

AhamX Bodhi

Where I Becomes Infinite

Top-K Sampling

Module 3.1: Working with LLM APIs

Prof. Sashikumaar Ganesan | IISc Bengaluru | 2026





Session Overview

Prerequisites

- Token probability distributions and softmax
- Temperature parameter and its effect on logits
- Concept of greedy vs. stochastic decoding
- Basic LLM API structure (`model`, `messages`)

What You Will Learn

- The top-k algorithm, step by step
- Why top-k was introduced and what problem it solves
- How k value affects output quality and diversity
- Strengths and limitations versus top-p
- When to use top-k and how to tune it



What is Top-K Sampling?

Definition

Top-K sampling restricts token generation to the k tokens with the highest probability at each step. All other tokens in the vocabulary are assigned zero probability. A token is then sampled from the renormalised distribution over only these k candidates. The parameter k is a fixed integer set before generation begins.

Intuition: A Shortlist at Every Step

- Imagine a vocabulary of 100,000 tokens — most are irrelevant at any given position. Top-k creates a shortlist of the k most plausible candidates
- By eliminating the long tail of improbable tokens, top-k prevents the model from ever accidentally sampling a bizarre, out-of-distribution token
- The shortlist has the same size k regardless of the model's confidence — this is the defining property (and key limitation) of top-k



The Top-K Algorithm --- Step by Step

Four Steps Applied at Every Generation Position

1. **Apply temperature:** Compute temperature-scaled probabilities $p_i(T) = \exp(z_i/T) / \sum_j \exp(z_j/T)$ over all V vocabulary tokens
2. **Sort descending:** Rank all V tokens by probability from highest to lowest:
 $p_{(1)} \geq p_{(2)} \geq \dots \geq p_{(V)}$
3. **Truncate:** Keep only the top k tokens; set $p_i = 0$ for all tokens with rank $> k$
4. **Re-normalise and sample:** Divide the kept probabilities by their sum so they add up to 1.0; draw one token from this distribution



Worked Example --- Top-K=3

Vocabulary of 6 Tokens, K=3

After temperature scaling, suppose probabilities are (sorted):

Token	Raw Prob.	After K=3	Re-normalised
"the"	0.40	0.40	0.533
"a"	0.30	0.30	0.400
"this"	0.05	0.05	0.067
"my"	0.12	0.00	0.000
"some"	0.08	0.00	0.000
"their"	0.05	0.00	0.000

Wait — "my" had 12% probability but is excluded because it falls outside the top 3. This illustrates top-k's key limitation: it may exclude plausible tokens when k is small.



Effect of the K Value on Output

K Value	Label	Output Character	Use Case
1	Greedy	Deterministic, identical every run	Exact reproduction, verification
5–10	Very tight	Focused, low diversity	Factual Q&A, structured extraction
20–40	Moderate	Coherent with variation	General chatbots, summarisation
40–50	Balanced	Good diversity, rare bad tokens	Creative writing assistance
50–100	Wide	High diversity	Brainstorming, open-ended generation
Disabled	No filter	Full vocabulary accessible	Relies entirely on temperature/top-p

Many models (GPT-2, early GPT-3) were released with top-k=40 as the default. Modern LLMs often default to top-k disabled, relying on top-p or temperature alone.



Top-K: Strengths and Limitations

Strengths of Top-K

- Simple and computationally efficient — just sort and truncate
- Eliminates rare, incoherent tokens from the vocabulary tail
- Gives developers direct, interpretable control ("top 40 options")
- Works well when distribution is consistently spread across a moderate number of plausible tokens
- Easy to reason about: exactly k candidates at every step

Limitations of Top-K

- Fixed k does not adapt to distribution shape
- When model is confident: $k = 50$ includes many implausible tokens
- When model is uncertain: $k = 50$ may exclude valid alternatives
- No principled way to choose k without empirical tuning
- Introduced in GPT-2 era; top-p has largely superseded it for general text generation tasks

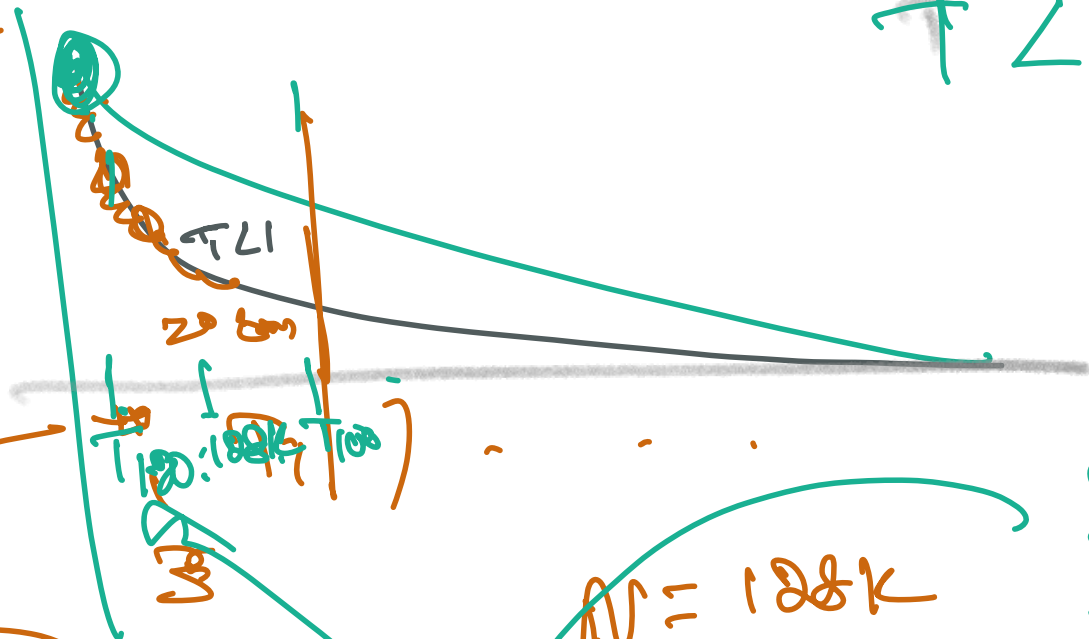
Comparing the Three Sampling Strategies



Property	Temperature	Top-K	Top-P
Mechanism	Rescales logits before softmax	Cuts off after K tokens by rank	Cuts off by cumulative probability mass
Fixed window?	No — reshapes full distribution	Yes — always K tokens	No — adapts to shape
Adapts to confidence?	Partially	No	Yes
Intuition	"How creative?"	"How many options?"	"How much probability mass?"
Best for	Controlling overall creativity	Hard vocabulary cut-off	Adaptive, balanced generation
Default in OpenAI API	1.0	Not exposed directly	1.0
Typical value	0.0 – 1.5	20 – 50	0.7 – 0.95

$\delta p < 1$
 $\log - 1 + 40$

$T < 1$



$\{0.81, 0.82, \dots\}$
40

T_{k+1}

$U = \{T_1, \dots, T_{100K}\}$
 $P(T_1) \cdot P(T_{100}) \cdot P(T_{100K})$

$U = 100K$



GenAI Application: Google Gemini's Sampling Stack

Top-K as Part of Gemini's Generation Stack

- **Gemini API:** Uniquely exposes top-k as a first-class parameter (unlike OpenAI which does not expose it in the public Chat API)
- **Default configuration:** Gemini 1.5 Pro defaults to `top_k=64` alongside `top_p=0.95`, applying both in sequence: top-k first, then top-p within the kept set
- **Gemini Flash:** Uses a smaller `top_k` (typically 32–40) tuned for faster, lower-latency generation with slightly less diversity
- **Vertex AI:** Enterprise deployments allow overriding all sampling parameters; ML teams typically reduce top-k for domain-specific applications requiring tight vocabulary control
- **Practical effect:** With `top_k=64` and `top_p=0.95`, the effective candidate set is often 20–30 tokens per step for normal conversational text

Google's approach of stacking top-k before top-p is common in production open-source serving.



GenAI Application: Open-Source LLM Inference

Ollama and llama.cpp Defaults

- **llama.cpp:** Exposes top-k with default $k = 40$ across all GGUF models (Llama 3, Mistral, Phi-3)
- **Ollama:** Uses top-k=40 as default; users override per-model in Modelfile
- **LM Studio:** Provides a slider (k=1 to 200) making top-k behavior directly observable
- Top-k=40 was established as a practical default through empirical studies on GPT-2 quality metrics

Hugging Face Transformers

- `model.generate()` exposes `top_k` parameter directly; disabling with $k = 0$ enables the full vocabulary for sampling
- Text Generation Inference (TGI) server uses top-k alongside top-p; logs rejected tokens for analysis and debugging
- Research teams study *token diversity* as a function of k to evaluate generation quality and detect repetition failures early



GenAI Application: Dialogue and Conversational AI

Top-K Tuning for Conversational Quality

- **Character.ai:** Uses moderate top-k (30–50) combined with higher temperature to produce varied, engaging dialogue without generating completely off-topic tokens that break character immersion
- **Customer service bots:** Low top-k (10–20) constrains responses to domain-relevant vocabulary; reduces risk of the model “wandering” into topics outside the product knowledge base
- **Mental health chatbots (Woebot, Wysa):** Very low top-k combined with safety fine-tuning ensures responses stay within clinically reviewed language patterns and avoid potentially harmful vocabulary
- **Interactive fiction (AI Dungeon):** High top-k (100+) and high temperature deliberately enable surprising, creative plot directions not achievable with constrained sampling



GenAI Application: Code Generation Models

Top-K for Code Completion

- **StarCoder (BigCode):** Default top-k=50 for multi-line code completion; lower values cause repetitive variable names
- **CodeLlama:** top-k=10–20 for precise function signature completion; top-k=50 for longer, exploratory code generation
- Top-k effectively enforces “stay in the programming language vocabulary” without explicit language filtering

Why Code Benefits from Top-K

- At any code position, only a small set of syntactically valid tokens are plausible
- A keyword like `if`, `for`, or a variable name each represent valid candidates; random English words are not
- Low top-k acts as a soft syntax filter without requiring grammar-constrained decoding
- Combined with low temperature: near-deterministic, syntactically clean code output

Session Summary



Key Takeaways

1. **Top-K sampling** keeps only the k most probable tokens at each step, setting all others to zero probability
2. The window is **fixed at size** k regardless of the model's confidence — this is the key difference from top-p
3. Typical values: $k = 1$ (greedy), $k = 40$ (GPT-2 default), $k = 50$ – 100 for creative tasks
4. Top-k is applied *after* temperature scaling and *before* or alongside top-p in stacked configurations
5. Top-k is a simpler, computationally cheaper filter; top-p is more principled for general text generation
6. Google Gemini and llama.cpp expose top-k prominently; OpenAI omits it from the public Chat API in favour of top-p

AhamX Bodhi

Where I Becomes Infinite